

—— // ELISPRS // ——

elisprs – Language Reference

v0.1.6

LANGUAGE REFERENCE

every builtin · special form · macro · variable

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

Special Forms	2
Arithmetic & Numbers	5
Predicates & Type Tests	11
Cons, Lists & Sequences	15
Symbols, Cells & Binding	18
Strings & Characters	20
I/O, Print & Format	24
Control & Functional	26
Association Lists & Hash Tables	28
Other	30

Special Forms

18 entries

quote

```
(quote OBJECT)
```

Return OBJECT, unevaluated.

function

```
(function OBJECT)
```

Like quote, but for functions (#').

lambda

```
(lambda ARGLIST BODY...)
```

An anonymous function.

progn

```
(progn BODY...)
```

Evaluate BODY in order; return the last value.

if

```
(if COND THEN ELSE...)
```

If COND is non-nil, eval THEN, else the ELSE forms.

when

```
(when COND BODY...)
```

If COND is non-nil, eval BODY.

unless

```
(unless COND BODY...)
```

If COND is nil, eval BODY.

and

```
(and CONDITIONS...)
```

Eval forms until one is nil; return the last value.

or

```
(or CONDITIONS...)
```

Eval forms until one is non-nil; return it.

setq

```
(setq SYM VAL SYM VAL...)
```

Set each SYM's value cell to VAL.

let

```
(let BINDINGS BODY...)
```

Bind variables in parallel, then eval BODY.

let*

```
(let* BINDINGS BODY...)
```

Bind variables sequentially, then eval BODY.

while

```
(while COND BODY...)
```

While COND is non-nil, eval BODY.

cond

```
(cond CLAUSES...)
```

Try each (TEST BODY...) clause; eval the first whose TEST is non-nil.

defun

```
(defun NAME ARGLIST BODY...)
```

Define NAME as a function.

defmacro

```
(defmacro NAME ARGLIST BODY...)
```

Define NAME as a macro.

defvar

```
(defvar NAME &optional INIT DOC)
```

Define a special (dynamic) variable.

defconst

```
(defconst NAME INIT &optional DOC)
```

Define a constant special variable.

Arithmetic & Numbers

46 entries

+

```
( + &rest NUMBERS )
```

Sum of the arguments.

-

```
( - &rest NUMBERS )
```

Negation, or subtraction from the first.

*

```
( * &rest NUMBERS )
```

Product of the arguments.

/

```
( / DIVIDEND &rest DIVISORS )
```

Quotient of the arguments.

%

```
( % X Y )
```

Integer remainder of X divided by Y.

1+

```
( 1+ NUMBER )
```

NUMBER plus one.

1-

```
( 1- NUMBER )
```

NUMBER minus one.

=

```
( = &rest NUMBERS )
```

Non-nil if all numbers are equal.

<

```
(< &rest NUMBERS)
```

Non-nil if numbers strictly increase.

>

```
(> &rest NUMBERS)
```

Non-nil if numbers strictly decrease.

<=

```
(<= &rest NUMBERS)
```

Non-nil if numbers are non-decreasing.

>=

```
(>= &rest NUMBERS)
```

Non-nil if numbers are non-increasing.

mod

```
(mod X Y)
```

X modulo Y, the remainder taking the sign of the divisor Y.

floor

```
(floor ARG &optional DIVISOR)
```

Largest integer less than or equal to ARG (or ARG/DIVISOR).

ceiling

```
(ceiling ARG &optional DIVISOR)
```

Smallest integer greater than or equal to ARG (or ARG/DIVISOR).

round

```
(round ARG &optional DIVISOR)
```

ARG (or ARG/DIVISOR) rounded to the nearest integer.

truncate

```
(truncate ARG &optional DIVISOR)
```

ARG (or ARG/DIVISOR) truncated toward zero to an integer.

float

```
(float ARG)
```

Convert ARG to a floating-point number.

logand

```
(logand &rest INTS)
```

Bitwise AND of the integer arguments.

logior

```
(logior &rest INTS)
```

Bitwise inclusive OR of the integer arguments.

logxor

```
(logxor &rest INTS)
```

Bitwise exclusive OR of the integer arguments.

lognot

```
(lognot NUMBER)
```

Bitwise NOT (one's complement) of NUMBER.

ash

```
(ash VALUE COUNT)
```

Arithmetic shift VALUE left COUNT bits (right if COUNT is negative).

lsh

```
(lsh VALUE COUNT)
```

Logical shift VALUE left COUNT bits (right if COUNT is negative).

expt

```
(expt BASE EXPONENT)
```

Return BASE raised to the power EXPONENT.

sqrt

```
(sqrt ARG)
```

Return the square root of ARG.

exp

```
(exp ARG)
```

Return e (the base of natural logarithms) raised to ARG.

log

```
(log ARG &optional BASE)
```

Natural logarithm of ARG, or the logarithm base BASE.

sin

```
(sin ARG)
```

Return the sine of ARG (in radians).

cos

```
(cos ARG)
```

Return the cosine of ARG (in radians).

tan

```
(tan ARG)
```

Return the tangent of ARG (in radians).

asin

```
(asin ARG)
```

Return the inverse sine of ARG, in radians.

acos

```
(acos ARG)
```

Return the inverse cosine of ARG, in radians.

atan

```
(atan Y &optional X)
```

Arc tangent of Y, or of Y/X using both signs to choose the quadrant.

ldexp

```
(ldexp SGNFCAND EXPONENT)
```

Return SGNFCAND * 2**EXPONENT, as a float.

copysign

```
(copysign X1 X2)
```

Return X1 with the sign of X2.

frexp

```
(frexp X)
```

Return (SIGNIFICAND . EXPONENT) with $X = \text{SIGNIFICAND} * 2^{\text{EXPONENT}}$ and $0.5 \leq |\text{SIGNIFICAND}| < 1$.

fround

```
(fround X)
```

Round X to the nearest integer, returned as a float.

ffloor

```
(ffloor X)
```

Largest integer value not greater than X, returned as a float.

fceiling

```
(fceiling X)
```

Smallest integer value not less than X, returned as a float.

ftruncate

```
(ftruncate X)
```

Truncate X toward zero to an integer value, returned as a float.

abs

```
(abs ARG)
```

Return the absolute value of ARG.

logcount

```
(logcount VALUE)
```

Number of one bits in VALUE (or zero bits if VALUE is negative).

logb

```
(logb ARG)
```

Return the binary exponent of ARG: the floor of log base 2 of |ARG|.

random

```
(random &optional LIMIT)
```

Pseudo-random integer; if LIMIT is a positive integer, in the range [0, LIMIT).

string-to-number

```
(string-to-number STRING &optional BASE)
```

Parse the number at the front of STRING (radix BASE, default 10).

Predicates & Type Tests

27 entries

eq

```
(eq A B)
```

Non-nil if A and B are the same object.

eq1

```
(eq1 A B)
```

Like eq, but compares numbers by value.

equal

```
(equal A B)
```

Non-nil if A and B are structurally equal.

null

```
(null OBJECT)
```

Non-nil if OBJECT is nil.

not

```
(not OBJECT)
```

Non-nil if OBJECT is nil.

consp

```
(consp OBJECT)
```

Non-nil if OBJECT is a cons cell.

listp

```
(listp OBJECT)
```

Non-nil if OBJECT is a list (cons or nil).

atom

```
(atom OBJECT)
```

Non-nil if OBJECT is not a cons cell.

symbolp

```
(symbolp OBJECT)
```

Non-nil if OBJECT is a symbol.

stringp

```
(stringp OBJECT)
```

Non-nil if OBJECT is a string.

numberp

```
(numberp OBJECT)
```

Non-nil if OBJECT is a number.

integerp

```
(integerp OBJECT)
```

Non-nil if OBJECT is an integer.

floatp

```
(floatp OBJECT)
```

Non-nil if OBJECT is a float.

vectorp

```
(vectorp OBJECT)
```

Non-nil if OBJECT is a vector.

zerop

```
(zerop NUMBER)
```

Non-nil if NUMBER is zero.

isnan

```
(isnan X)
```

Non-nil if the float X is a NaN.

type-of

```
(type-of OBJECT)
```

Return a symbol naming the primitive type of OBJECT.

recordp

```
(recordp OBJECT)
```

Non-nil if OBJECT is a record.

cl-struct-p

```
(cl-struct-p OBJECT)
```

Non-nil if OBJECT is a record (a cl-defstruct instance).

functionp

```
(functionp OBJECT)
```

Non-nil if OBJECT is callable as a function.

char-or-string-p

```
(char-or-string-p OBJECT)
```

Non-nil if OBJECT is a character (integer) or a string.

subrp

```
(subrp OBJECT)
```

Non-nil if OBJECT is a built-in (primitive) function.

macrop

```
(macrop OBJECT)
```

Non-nil if OBJECT is a macro.

special-form-p

```
(special-form-p OBJECT)
```

Non-nil if OBJECT is a special form.

char-uppercase-p

```
(char-uppercase-p CHAR)
```

Non-nil if CHAR is an uppercase character.

special-variable-p

```
(special-variable-p SYMBOL)
```

Non-nil if SYMBOL is a special (dynamically scoped) variable.

char-equal

```
(char-equal C1 C2)
```

Non-nil if characters C1 and C2 are equal (case-insensitive when case-fold-search).

Cons, Lists & Sequences

18 entries

cons

```
(cons CAR CDR)
```

Create a new cons cell.

car

```
(car LIST)
```

Return the first element of LIST.

cdr

```
(cdr LIST)
```

Return the rest of LIST after the first element.

setcar

```
(setcar CELL VALUE)
```

Set the car of CELL to VALUE.

setcdr

```
(setcdr CELL VALUE)
```

Set the cdr of CELL to VALUE.

list

```
(list &rest OBJECTS)
```

Return a newly created list of OBJECTS.

append

```
(append &rest SEQUENCES)
```

Concatenate lists into one.

reverse

```
(reverse SEQUENCE)
```

Return a reversed copy of SEQUENCE.

length

```
(length SEQUENCE)
```

Return the length of SEQUENCE.

nth

```
(nth N LIST)
```

Return the Nth element of LIST.

vector

```
(vector &rest OBJECTS)
```

Return a vector of OBJECTS.

make-vector

```
(make-vector LENGTH INIT)
```

Return a vector of LENGTH elements, all INIT.

aref

```
(aref ARRAY IDX)
```

Return the IDX'th element of ARRAY.

aset

```
(aset ARRAY IDX VALUE)
```

Set the IDX'th element of ARRAY to VALUE.

fillarray

```
(fillarray ARRAY ITEM)
```

Set every element of ARRAY to ITEM; return ARRAY.

vconcat

```
(vconcat &rest SEQUENCES)
```

Concatenate the SEQUENCES into a single new vector.

string-to-vector

```
(string-to-vector STRING)
```

Return a vector of the character codes in STRING.

string-to-list

```
(string-to-list STRING)
```

Return a list of the character codes in STRING.

Symbols, Cells & Binding

12 entries

symbol-name

```
(symbol-name SYMBOL)
```

Return SYMBOL's name as a string.

intern

```
(intern NAME)
```

Return the interned symbol named NAME.

make-symbol

```
(make-symbol NAME)
```

Return a fresh symbol named NAME.

set

```
(set SYMBOL VALUE)
```

Set SYMBOL's value cell to VALUE.

symbol-value

```
(symbol-value SYMBOL)
```

Return SYMBOL's value.

boundp

```
(boundp SYMBOL)
```

Non-nil if SYMBOL has a value.

fboundp

```
(fboundp SYMBOL)
```

Non-nil if SYMBOL has a function definition.

fset

```
(fset SYMBOL DEFINITION)
```

Set SYMBOL's function cell to DEFINITION.

makunbound

```
(makunbound SYMBOL)
```

Make SYMBOL's value cell void; return SYMBOL.

intern-soft

```
(intern-soft NAME)
```

Return the interned symbol named NAME, or nil if none exists.

symbol-function

```
(symbol-function SYMBOL)
```

Return SYMBOL's function definition, or nil if it has none.

indirect-function

```
(indirect-function OBJECT &optional NOERROR)
```

Chase symbol function cells from OBJECT to the underlying function.

Strings & Characters

27 entries

concat

```
(concat &rest STRINGS)
```

Concatenate STRINGS into one string.

number-to-string

```
(number-to-string NUMBER)
```

Return NUMBER rendered as a string.

string-match

```
(string-match REGEXP STRING &optional START)
```

Search STRING for REGEXP from START; set match data, return match index or nil.

string-match-p

```
(string-match-p REGEXP STRING &optional START)
```

Like string-match but preserve the existing match data.

match-beginning

```
(match-beginning N)
```

Char position where the Nth subexpression of the last match began.

match-end

```
(match-end N)
```

Char position where the Nth subexpression of the last match ended.

match-string

```
(match-string N &optional STRING)
```

Text matched by the Nth subexpression of the last match.

match-data

```
(match-data)
```

Last match's positions as a flat list (beg0 end0 beg1 end1 ...).

set-match-data

```
(set-match-data LIST)
```

Restore match positions from a match-data list.

replace-regexp-in-string

```
(replace-regexp-in-string REGEXP REP STRING &optional FIXEDCASE LITERAL SUBEXP START)
```

Replace every match of REGEXP in STRING with REP — a string (& / \N templates unless LITERAL) or a function of the matched text.

regexp-quote

```
(regexp-quote STRING)
```

Return STRING with regexp metacharacters escaped to match literally.

save-match-data

```
(save-match-data &rest BODY)
```

Eval BODY, preserving the caller's regexp match data.

substring

```
(substring STRING &optional FROM TO)
```

Return the substring of STRING from index FROM to TO (negative indices count from the end).

split-string

```
(split-string STRING &optional SEPARATORS OMIT-NULLS TRIM)
```

Split STRING into a list of substrings around matches of SEPARATORS.

string-prefix-p

```
(string-prefix-p PREFIX STRING &optional IGNORE-CASE)
```

Non-nil if PREFIX is a prefix of STRING.

string-suffix-p

```
(string-suffix-p SUFFIX STRING &optional IGNORE-CASE)
```

Non-nil if SUFFIX is a suffix of STRING.

string-empty-p

```
(string-empty-p STRING)
```

Non-nil if STRING is the empty string.

string-join

```
(string-join STRINGS &optional SEPARATOR)
```

Concatenate the list STRINGS, inserting SEPARATOR between elements.

char-to-string

```
(char-to-string CHAR)
```

Return a one-character string containing CHAR.

string-to-char

```
(string-to-char STRING)
```

Return the first character of STRING as an integer (0 if empty).

make-string

```
(make-string COUNT CHARACTER &optional MULTIBYTE)
```

Return a string of COUNT copies of CHARACTER.

string

```
(string &rest CHARACTERS)
```

Return a string made from the character arguments.

string-search

```
(string-search NEEDLE HAYSTACK &optional START-POS)
```

Index of the first occurrence of NEEDLE in HAYSTACK (from START-POS), or nil.

downcase

```
(downcase OBJ)
```

Return OBJ (a string or character) converted to lower case.

upcase

```
(upcase OBJ)
```

Return OBJ (a string or character) converted to upper case.

compare-strings

```
(compare-strings STR1 START1 END1 STR2 START2 END2 &optional IGNORE-CASE)
```

Compare the specified substrings; t if equal, else a signed mismatch position.

string-distance

```
(string-distance STRING1 STRING2 &optional BYTECOMPARE)
```

Levenshtein edit distance between STRING1 and STRING2.

I/O, Print & Format

11 entries

format

```
(format STRING &rest ARGS)
```

Format ARGS per the %-directives in STRING.

message

```
(message FORMAT &rest ARGS)
```

Print a formatted message (to stderr).

princ

```
(princ OBJECT)
```

Output OBJECT (no quoting).

prin1

```
(prin1 OBJECT)
```

Output OBJECT in read syntax.

read

```
(read STRING)
```

Read and return one Lisp object from STRING.

read-from-string

```
(read-from-string STRING &optional START END)
```

Read one object from STRING; return (OBJECT . NEXT-INDEX).

terpri

```
(terpri &optional STREAM)
```

Output a newline.

print

```
(print OBJECT &optional STREAM)
```

Output OBJECT in read syntax, surrounded by newlines; return OBJECT.

prin1-to-string

```
(prin1-to-string OBJECT)
```

Return a string of OBJECT printed in read syntax.

--push-output-capture--

```
(--push-output-capture--)
```

Internal: start redirecting princ/prin1/print output into a capture buffer.

--pop-output-capture--

```
(--pop-output-capture--)
```

Internal: stop capturing output and return the captured string.

Control & Functional

11 entries

eval

```
(eval FORM &optional LEXICAL)
```

Evaluate FORM and return its value.

funcall

```
(funcall FUNCTION &rest ARGS)
```

Call FUNCTION with ARGS.

apply

```
(apply FUNCTION &rest ARGS LIST)
```

Call FUNCTION with ARGS and the elements of LIST.

identity

```
(identity ARG)
```

Return ARG unchanged.

pcase

```
(pcase EXPR (PATTERN BODY...)...)
```

Structural dispatch: `_`, literals, `'x`, binders, (pred FN), (guard E), (and ...), (or ...).

func-arity

```
(func-arity FUNCTION)
```

Return (MIN . MAX) giving FUNCTION's minimum and maximum argument counts (MAX may be 'many').

subr-arity

```
(subr-arity SUBR)
```

Return (MIN . MAX), the minimum and maximum argument counts of built-in SUBR.

throw

```
(throw TAG VALUE)
```

Throw to the 'catch' for TAG, returning VALUE from it.

error

```
(error FORMAT &rest ARGS)
```

Signal an error whose message is FORMAT formatted with ARGS.

user-error

```
(user-error FORMAT &rest ARGS)
```

Signal a user error (a mistake, not a bug) with a formatted message.

signal

```
(signal ERROR-SYMBOL DATA)
```

Signal the error named ERROR-SYMBOL with associated DATA.

Association Lists & Hash Tables

11 entries

make-hash-table

```
(make-hash-table &rest KEYWORD-ARGS)
```

Create and return a new empty hash table (accepts :test, :size, etc.).

gethash

```
(gethash KEY TABLE &optional DEFAULT)
```

Return the value for KEY in TABLE, or DEFAULT if absent.

puthash

```
(puthash KEY VALUE TABLE)
```

Associate KEY with VALUE in TABLE; return VALUE.

remhash

```
(remhash KEY TABLE)
```

Remove KEY and its value from TABLE.

clrhash

```
(clrhash TABLE)
```

Remove all entries from TABLE; return it.

hash-table-count

```
(hash-table-count TABLE)
```

Return the number of entries in TABLE.

hash-table-test

```
(hash-table-test TABLE)
```

Return the test function symbol used by TABLE.

hash-table-p

```
(hash-table-p OBJECT)
```

Non-nil if OBJECT is a hash table.

hash-table-keys

```
(hash-table-keys TABLE)
```

Return a list of all keys in TABLE.

hash-table-values

```
(hash-table-values TABLE)
```

Return a list of all values in TABLE.

copy-hash-table

```
(copy-hash-table TABLE)
```

Return a shallow copy of TABLE.

Other

89 entries

format-time-string

```
(format-time-string FORMAT &optional TIME ZONE)
```

Format TIME (default now) per FORMAT. ZONE nil=local, t=UTC, integer=offset secs.

float-time

```
(float-time &optional TIME)
```

Return TIME (default now) as seconds since the epoch, a float.

current-time

```
(current-time)
```

Return the current time as (HIGH LOW USEC PSEC).

current-time-string

```
(current-time-string &optional TIME ZONE)
```

Return TIME (default now) as a string like "Sun Jun 29 12:00:00 2025".

decode-time

```
(decode-time &optional TIME ZONE)
```

Decompose TIME into (SEC MIN HOUR DAY MON YEAR DOW DST UTCOFF).

encode-time

```
(encode-time TIME) or (encode-time SEC MIN HOUR DAY MON YEAR &optional ZONE)
```

Inverse of decode-time: turn decoded components into a (HIGH LOW) time value.

defvaralias

```
(defvaralias NEW-ALIAS BASE-VARIABLE &optional DOCSTRING)
```

Make NEW-ALIAS a variable alias for BASE-VARIABLE: value operations on either affect both. Returns BASE-VARIABLE.

indirect-variable

```
(indirect-variable OBJECT)
```

Follow the 'defvaralias' chain from OBJECT to the base variable symbol, or return OBJECT if it is not a symbol.

sha1

```
(sha1 OBJECT &optional START END BINARY)
```

Return the SHA-1 hash of OBJECT (a string or buffer) as a hex string.

md5

```
(md5 OBJECT &optional START END CODING-SYSTEM NOERROR)
```

Return the MD5 hash of OBJECT as a hex string.

secure-hash

```
(secure-hash ALGORITHM OBJECT &optional START END BINARY)
```

Return the ALGORITHM (md5/sha1/sha224/sha256/sha384/sha512) hash of OBJECT.

base64-encode-string

```
(base64-encode-string STRING &optional NO-LINE-BREAK)
```

Return the base64 encoding of STRING.

base64-decode-string

```
(base64-decode-string STRING &optional BASE64URL IGNORE-INVALID)
```

Return the contents of base64-encoded STRING decoded.

base64url-encode-string

```
(base64url-encode-string STRING &optional NO-PAD)
```

Return the base64url (URL-safe) encoding of STRING.

base64url-decode-string

```
(base64url-decode-string STRING &optional IGNORE-INVALID)
```

Decode a base64url (URL-safe) encoded STRING.

url-hexify-string

```
(url-hexify-string STRING &optional ALLOWED-CHARS)
```

Percent-encode (URL-escape) the characters of STRING.

url-unhex-string

```
(url-unhex-string STRING &optional ALLOW-NEWLINES)
```

Decode the %-escaped sequences in STRING.

sxhash

```
(sxhash OBJ)
```

Return an 'equal'-based hash code for OBJ (alias of sxhash-equal).

sxhash-equal

```
(sxhash-equal OBJ)
```

Return a hash code for OBJ such that 'equal' objects hash alike.

sxhash-eq

```
(sxhash-eq OBJ)
```

Return a hash code for OBJ such that 'eq' objects hash alike.

sxhash-eql

```
(sxhash-eql OBJ)
```

Return a hash code for OBJ such that 'eql' objects hash alike.

getenv

```
(getenv VARIABLE &optional FRAME)
```

Return the value of environment VARIABLE as a string, or nil.

setenv

```
(setenv VARIABLE &optional VALUE SUBSTITUTE-ENV-VARS)
```

Set environment VARIABLE to VALUE (unset it if VALUE is nil).

--current-directory--

```
(--current-directory--)
```

Internal: the process working directory as a string (backs 'default-directory').

file-exists-p

```
(file-exists-p FILENAME)
```

Non-nil if FILENAME names an existing file.

file-directory-p

```
(file-directory-p FILENAME)
```

Non-nil if FILENAME names an existing directory.

file-regular-p

```
(file-regular-p FILENAME)
```

Non-nil if FILENAME names a regular file.

file-readable-p

```
(file-readable-p FILENAME)
```

Non-nil if FILENAME names a file you can read.

file-writable-p

```
(file-writable-p FILENAME)
```

Non-nil if FILENAME names a file you can write (or create).

file-symlink-p

```
(file-symlink-p FILENAME)
```

If FILENAME is a symbolic link, return its target; else nil.

file-executable-p

```
(file-executable-p FILENAME)
```

Non-nil if FILENAME names a file you can execute (search permission for a directory).

--invocation-file--

```
(--invocation-file--)
```

Internal: absolute path of the running `elisp` binary, backing invocation-name', invocation-directory', and exec-directory'.

--directory-files--

```
(--directory-files-- DIRECTORY &optional MATCH-REGEXP NOSORT)
```

Internal primitive behind 'directory-files': names in DIRECTORY, optionally MATCH-filtered and sorted.

write-region

```
(write-region START END FILENAME &optional APPEND VISIT LOCKNAME MUSTBENEW)
```

Write the buffer text between START and END to FILENAME.

delete-file

```
(delete-file FILENAME &optional TRASH)
```

Delete the file named FILENAME.

make-directory

```
(make-directory DIR &optional PARENTS)
```

Create the directory DIR (and parents when PARENTS is non-nil).

rename-file

```
(rename-file FILE NEWNAME &optional OK-IF-ALREADY-EXISTS)
```

Rename FILE to NEWNAME.

copy-file

```
(copy-file FILE NEWNAME &optional OK-IF-ALREADY-EXISTS KEEP-TIME PRESERVE-UID-GID PRESERVE-PERMISSIONS)
```

Copy FILE to NEWNAME.

insert-file-contents

```
(insert-file-contents FILENAME &optional VISIT BEG END REPLACE)
```

Insert the contents of FILENAME into the current buffer at point.

shell-command-to-string

```
(shell-command-to-string COMMAND)
```

Run shell COMMAND and return its standard output as a string.

call-process

```
(call-process PROGRAM &optional INFILE DESTINATION DISPLAY &rest ARGS)
```

Run PROGRAM synchronously with ARGS; return its exit status.

process-lines

```
(process-lines PROGRAM &rest ARGS)
```

Run PROGRAM with ARGS and return its output as a list of lines.

insert

```
(insert &rest ARGS)
```

Insert the ARGS (strings or characters) into the buffer at point.

buffer-string

```
(buffer-string)
```

Return the entire contents of the current buffer as a string.

buffer-size

```
(buffer-size &optional BUFFER)
```

Return the number of characters in the current buffer.

point

```
(point)
```

Return the value of point, as an integer (1-based).

point-min

```
(point-min)
```

Return the minimum accessible buffer position.

point-max

```
(point-max)
```

Return the maximum accessible buffer position.

goto-char

```
(goto-char POSITION)
```

Set point to POSITION in the current buffer.

erase-buffer

```
(erase-buffer)
```

Delete the entire contents of the current buffer.

char-after

```
(char-after &optional POS)
```

Return the character after POS (point by default), or nil.

buffer-substring

```
(buffer-substring START END)
```

Return the buffer text between positions START and END as a string.

buffer-substring-no-properties

```
(buffer-substring-no-properties START END)
```

Return the buffer text between START and END, with no text properties.

delete-region

```
(delete-region START END)
```

Delete the buffer text between positions START and END.

forward-char

```
(forward-char &optional N)
```

Move point N characters forward (default 1).

backward-char

```
(backward-char &optional N)
```

Move point N characters backward (default 1).

beginning-of-line

```
(beginning-of-line &optional N)
```

Move point to the beginning of the current line.

end-of-line

```
(end-of-line &optional N)
```

Move point to the end of the current line.

line-beginning-position

```
(line-beginning-position &optional N)
```

Return the position of the beginning of the current line.

line-end-position

```
(line-end-position &optional N)
```

Return the position of the end of the current line.

pos-bol

```
(pos-bol &optional N)
```

Return the position of the beginning of the current line (like line-beginning-position).

pos-eol

```
(pos-eol &optional N)
```

Return the position of the end of the current line (like line-end-position).

bolp

```
(bolp)
```

Non-nil if point is at the beginning of a line.

eolp

```
(eolp)
```

Non-nil if point is at the end of a line.

bobp

```
(bobp)
```

Non-nil if point is at the beginning of the buffer.

eobp

```
(eobp)
```

Non-nil if point is at the end of the buffer.

forward-line

```
(forward-line &optional N)
```

Move point N lines forward; return the count of lines left unmoved.

search-forward

```
(search-forward STRING &optional BOUND NOERROR COUNT)
```

Search forward for STRING, leaving point after it; return the new point.

re-search-forward

```
(re-search-forward REGEXP &optional BOUND NOERROR COUNT)
```

Search forward for a match of REGEXP, leaving point after it.

looking-at

```
(looking-at REGEXP &optional INHIBIT-MODIFY)
```

Non-nil if text after point matches REGEXP; sets the match data.

looking-at-p

```
(looking-at-p REGEXP)
```

Non-nil if text after point matches REGEXP, without changing the match data.

replace-match

```
(replace-match NEWTEXT &optional FIXEDCASE LITERAL STRING SUBEXP)
```

Replace the text matched by the last search with NEWTEXT.

char-before

```
(char-before &optional POS)
```

Return the character before POS (point by default), or nil.

delete-char

```
(delete-char N &optional KILLFLAG)
```

Delete N characters forward from point (backward if N is negative).

insert-char

```
(insert-char CHARACTER &optional COUNT INHERIT)
```

Insert COUNT copies of CHARACTER at point.

count-lines

```
(count-lines START END)
```

Return the number of lines between positions START and END.

line-number-at-pos

```
(line-number-at-pos &optional POS ABSOLUTE)
```

Return the line number of POS (point by default) in the current buffer.

current-column

```
(current-column)
```

Return the horizontal column position of point.

search-backward

```
(search-backward STRING &optional BOUND NOERROR COUNT)
```

Search backward for STRING, leaving point before it.

re-search-backward

```
(re-search-backward REGEXP &optional BOUND NOERROR COUNT)
```

Search backward for a match of REGEXP, leaving point before it.

skip-chars-forward

```
(skip-chars-forward STRING &optional LIM)
```

Move point forward over characters in the set STRING; return the distance moved.

skip-chars-backward

```
(skip-chars-backward STRING &optional LIM)
```

Move point backward over characters in the set STRING; return the distance moved.

forward-word

```
(forward-word &optional N)
```

Move point forward N words (default 1).

backward-word

```
(backward-word &optional N)
```

Move point backward N words (default 1).

intercept

```
(intercept KIND PATTERN FORM)
```

Register AOP advice. KIND is before/after/around; PATTERN is a glob ("forward-*", "_*", "all") matched against function names; FORM is the (quoted) advice form. Returns the intercept ID. Distinct from nadvice's advice-add: one registration fires on every matching name.

intercept-list

```
(intercept-list)
```

Return registered intercepts as a list of (ID KIND PATTERN FORM) entries, or nil.

intercept-remove

```
(intercept-remove ID)
```

Remove the intercept with integer ID. Returns t if removed, nil otherwise.

intercept-clear

```
(intercept-clear)
```

Remove all intercepts. Returns the count removed.

intercept-proceed

```
(intercept-proceed)
```

From an around advice, run the original function and return its value. Bound context in advice: intercept-name, intercept-args, intercept-cmd, and (after) intercept-ms/intercept-us.